`scripts/firebase-distribute.sh` — User Guide

Last verified: 2026-08-26 (LVA-120 combined mode RETIRED; §6.AA staging gate now unconditional; LVA-148/149 axis rename — `channel` split into `app` + `build_variant`) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate)

Overview

Stub doc generated from the script's in-source header comment. See `scripts/firebase-distribute.sh` for canonical behavior. This stub exists so the `CM-SCRIPT-DOCS-SYNC` pre-push gate (`.githooks/pre-push` Check 9) starts gating modifications to this script.

The script's own header documentation (verbatim):

```
scripts/firebase-distribute.sh — upload built artifacts to
Firebase App
Distribution and invite testers loaded from .env.

Replaces the local releases/ delivery flow as the canonical
operator
distribution channel (operator directive 2026-05-05).

Usage:
  ./scripts/firebase-distribute.sh                    # debug +
release APKs
  ./scripts/firebase-distribute.sh --debug-only       # only debug
APK
  ./scripts/firebase-distribute.sh --release-only     # only
release APK
  ./scripts/firebase-distribute.sh --release-notes "<text>"   #
custom notes

Inputs:
  .env  (gitignored) — LAVA_FIREBASE_TOKEN, project + app IDs,
tester emails
  releases/<version>/android-debug/*.apk
  releases/<version>/android-release/*.apk

Outputs:
  App Distribution release at the Firebase Console under
```

```
    project $LAVA_FIREBASE_PROJECT_ID, app
$LAVA_FIREBASE_ANDROID_APP_ID.
  3 testers receive an email invite (per .env
LAVA_FIREBASE_TESTERS_*).

Constitutional bindings:
  §6.H Credential Security — tokens read from .env, never echoed
  §6.J Anti-Bluff — propagates real failures via set -euo
pipefail; no WARN swallow
  §6.G End-to-end provider operational verification — distribute
step is the
        hand-off the operator's manual real-device pass exercises
against.
```

Usage

See the script's in-source comment block (above) for canonical usage examples.

Phase 1 Gate 7 (§6.AK cycle-coverage) — added 2026-06-26

Closes the firebase-distribute portion of §6.AK-debt. After the §6.P/§6.AA/§6.Z Phase-1 gates, the script invokes `scripts/check-cycle-coverage.sh` for the version being distributed (`--evidence-dir="$CHANGELOG_DIR"`, `--strict`; `--channel` is no longer passed — see the LVA-149 note below) and **refuses the distribute** unless EVERY CHANGELOG-claimed user-visible fix has an EXECUTED+PASSED covering device Challenge in the §6.Z evidence file for the SAME commit SHA. This is the gate that would have caught the 1076 incident (`627a0d58`: a C00-only device gate while the CHANGELOG claimed search / provider-selection / onboarding fixes). Exit mapping: `0` PASS · `1` an uncovered claim · `2` evidence/map missing/stale/wrong-SHA. The `|| ak_rc=$?` idiom keeps `set -e` from aborting before the §6.AK FATAL directive prints. Gates BOTH apps (client + api-app) via the app-resolved `$CHANGELOG_DIR`. Companion hermetic test: `tests/cycle-coverage/test_wiring.sh` (5/5, mutation-rehearsal proven); the gate's own test is `tests/cycle-coverage/test_cycle_coverage.sh` (7/7). The cycle author must write `<vname>-<code>-test-evidence.{md,json}` (with the `cycle-coverage:` header + per-Challenge `challenge:` rows) and `<vname>-<code>-cycle-coverage-map.yaml` under the channel dir for this gate to pass on a real distribute.

LVA-019 coverage-ledger snapshot — added 2026-08-20

Closes the firebase-distribute portion of LVA-019 (§11.4.25 per-release coverage ledger). Immediately after the §6.AK cycle-coverage Gate 7 block, the script invokes scripts/snapshot-coverage-ledger.sh "$APP_VERSION-$APP_VERSION_CODE", which regenerates docs/coverage-ledger.yaml (via scripts/generate-coverage-ledger.sh --quiet) and copies the freshly-generated file to .lava-ci-evidence/coverage-ledger-snapshots/<version>-<code>.yaml — a permanent, per-release historical record of coverage-ledger state at the exact moment that version was distributed. This mirrors the existing <vname>-<code>-cycle-coverage-map.yaml per-release snapshot pattern already used by the §6.AK gate above it.

The step is **advisory, not a release gate** — a snapshot failure logs a WARNING and the distribute continues, because the coverage ledger's own STRICT gate (scripts/check-coverage-ledger.sh --strict, wired into scripts/verify-all-constitution-rules.sh) already enforces ledger health elsewhere in local CI; this step exists purely to freeze a historical copy, not to re-litigate ledger correctness at distribute time. Companion script: scripts/snapshot-coverage-ledger.sh (documented separately). Bluff-Audit rehearsal: the snapshot mechanism was smoke-tested during Task 4 of the 2026-08-20 workable-items-backlog-closure plan — a 0.0.0-test snapshot was generated, diffed byte-for-byte against the live docs/coverage-ledger.yaml, confirmed identical, then removed; this proves the snapshot step correctly captures ledger state as of the wiring commit rather than a stale or hand-authored copy.

2026-08-26 — LVA-120: the combined --debug-and-release mode is RETIRED

The defect

MODE had three reachable values: debug, release, and both. The combined mode uploaded the **R8-minified RELEASE APK**, and in that mode:

- the **§6.AA staging gate never evaluated** — it was guarded on MODE == "release", and both is not release; and
- the **§6.AK / §6.Z device-evidence gate resolved to the DEBUG build variant** — the gate's variable was set by case "$MODE" in release) ... ;; *) debug ;; esac, and both fell into the catch-all.

That is mechanically the same setup as the **1.2.19-1039 forensic anchor** that birthed §6.AA in the first place: a release APK distributed without release-variant verification, crashing every cold launch through an R8-specific `painterResource` rejection. The mode that existed to be convenient reproduced the failure §6.AA was written to prevent.

Why removal, not another gate

A gate bolted onto the combined mode would have had to re-derive, at that one call site, everything the two-stage path already establishes by construction. The two-stage path evaluates both gates against the **correct build variant** because each stage *is* a single variant — so removing the mode makes every caller inherit the fix, with no third code path to keep in lockstep. This was the operator-approved remedy **[B]**.

The flag fails loudly rather than disappearing

`--debug-and-release` and `--both` are still parsed — and immediately **exit 1** with a FATAL §6.AA message naming the two-stage flow. This is deliberate. Deleting the branch outright would let the flag fall through to the argument parser's `*) shift` arm and silently become a **debug-only** distribute: a caller asking for release would get no release *and no error*, which is worse than the defect being removed.

```
FATAL §6.AA: '--both' is RETIRED (LVA-120). There is no combined distribute mode.
       It uploaded the release APK while the §6.AA staging gate never evaluated
       and the §6.AK/§6.Z device-evidence gate checked DEBUG-variant evidence.
       Use the §6.AA two-stage flow instead — it is now the ONLY path:
         1. ... --debug-only   # stage 1: distribute debug, then obtain
                               #          operator verification on the
                               #          failure-surface device class
         2. ... --release-only # stage 2: ONLY after that written confirmation
```

`MODE` now has exactly two reachable values, and an unexpected one is FATAL at three separate points (the Gate-1 pointer selection, the §6.AK gate invocation, and the pointer-persist step) rather than defaulting silently to the weaker variant.

> **Note on the in-script header.** The `# Usage:` block quoted verbatim earlier in this document still shows `./scripts/firebase-distribute.sh   # debug + release APKs`. That line is **stale**: the bare invocation has defaulted to debug-only since the §6.AA default flip, and combined mode no longer exists at all. Trust this section over the quoted header.

The §6.AA staging gate is now unconditional

The release-stage check previously read:

```
if [[ "$MODE" == "release" && -f "$LAST_VERSION_DEBUG_FILE" ]];
      then
```

The second conjunct meant a build variant with **no debug pointer** skipped the staging check entirely — the same class of hole as the retired combined mode, because *a gate that does not evaluate is not a gate*. The condition is now `MODE == "release"` alone; an absent pointer reads as `0`, which correctly fails, since no debug stage has run.

The failure message no longer offers the combined-mode escape hatch it used to list as option (b). The only remedy printed is the real one: run `--debug-only`, obtain operator verification of the **Firebase-distributed** debug build on the failure-surface device class, then re-run with `--release-only`.

2026-08-26 — LVA-148 / LVA-149: the `channel` axis, split and named

The problem

One word named two different things in artifacts that are read together:

Axis	What it selects	What it used to be called
A	Which **application** — `client` vs `api-app`	`channel` (in `contracts/distribution-record.schema.json`, and `CHANGELOG_CHANNEL` here)
B	Which **build variant** — `debug` vs `release`	`channel` (in `AK_CHANNEL` here, and `check-cycle-coverage.sh --channel`)

Two axes sharing one word, in the repair path for §6.AK, is what LVA-148 records. The contract's property set is now `app` (axis A) + `build_variant` (axis B), and **neither is called `channel`**.

What changed in this script

- **`AK_CHANNEL` is gone.** LVA-149 found that `check-cycle-coverage.sh --channel` *selected nothing* — that gate resolves both the cycle-coverage map and the §6.Z evidence from `--evidence-dir` + `--version` alone, so `debug`, `release`, or anything else produced byte-identical results. Passing it now fails loudly there, so it is no longer passed from here, and the variable it fed went with it. Axis B still selects which artifact is built and uploaded — it just never gated §6.AK.
- **The `MODE` validation at that call site is retained on its own merit.** LVA-120 recorded that a silent `*)` catch-all at exactly this point is what pointed a release distribute at debug-variant evidence. An unexpected `MODE` must still fail loudly, so the `case` remains — it validates rather than assigns.
- **`CHANGELOG_CHANNEL` deliberately keeps its legacy name.** It carries **axis A** (the application). Renaming it here alone would break a live gate: it is a *textual* contract — `scripts/pipeline/phase-05a-changelog-entry.sh` greps this file for the literal `CHANGELOG_CHANNEL="<value>"`, and `tests/pipeline/test_phase_05a_per_app_raw_evidence.sh` + `tests/pipeline/test_phase_05a_snapshot_claim.sh` mirror the same literal. Renaming it to `CHANGELOG_APP` across all four call sites at once is the **owed follow-up**, recorded rather than half-done.
- **Comment vocabulary swept.** The per-variant pointer machinery (`last-version-debug` / `last-version-release`) is now described as *per-build-variant* throughout, not *per-channel*, so the two axes read distinctly in this file.

Pointer persistence

The `both` arm — which wrote all three pointers at once — is removed. `debug` advances `last-version-debug`, `release` advances `last-version-release`, and each then refreshes the legacy variant-agnostic `last-version` to the **higher** of the two, so `scripts/tag.sh` and other downstream readers still see "latest distributed at all". An unexpected `MODE` at this point is FATAL rather than a silent no-write.

Modes (current, authoritative)

Invocation	Effect
(no flags)	Stage 1 — debug APK only (`MODE=debug`, the default since the §6.AA flip)
`--debug-only`	Stage 1 — debug APK only
`--release-only`	Stage 2 — release APK only; **refuses** unless `last-version-debug >=` this versionCode

Invocation	Effect
`--debug-and-release` / `--both`	**RETIRED — exit 1** with the two-stage remedy (LVA-120)
`--app client|api-app`	Axis A — selects which application's artifacts and evidence directory are used
`--release-notes "<text>"`	Override the release notes injected into App Distribution

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC requires it). Per §11.4.18, the documentation MUST stay in sync with the codebase — no doc may be out of sync with its script.

Cross-references

- `scripts/firebase-distribute.sh` — the script itself
- `docs/helix-constitution-gates.md` — gate inventory
- HelixConstitution `Constitution.md` §11.4.18 (the mandate)
- Lava `CLAUDE.md` §6.AD (HelixConstitution Inheritance)